

Chapitre 16 : Terminaison et correction d'algorithmes

ITC - 2025/2026

Félix POTIÉ

Introduction	3
Spécification	4
Terminaison	6
Correction partielle itératifs	14
Correction des algorithmes récursifs	19

Introduction

Maintenant que nous avons vu comment coder des algorithmes, nous allons voir comment s'assurer qu'ils fonctionnent correctement.

Spécification

Définition 1: Spécification de fonction.

La spécification d'un algorithme est une description de ce qu'il doit faire.

En général, on utilise :

- des **pré-conditions (entrées)** sur les paramètres de la fonction
- des **post-conditions (sorties)** sur ce que doit retourner/modifier la fonction

Avoir une spécification claire nous permettra de prouver la terminaison et la correction de l'algorithme.

Spécification - 2

```
def pgcd(a, b):
```

```
    """
```

```
    Entrées :
```

```
        a et b sont des entiers positifs
```

```
    Sortie :
```

```
        retourne le plus grand diviseur commun de a et b
```

```
    """
```

```
        while min(a,b) > 0:
```

```
            if a < b:
```

```
                b = b-a
```

```
            else:
```

```
                a = a-b
```

```
        return max(a,b)
```

Terminaison

La première chose à vérifier est que l'algorithme termine.

On vérifie que chaque boucle termine, que chaque appel récursif termine, et que chaque appel à d'autres fonctions termine.

Définition 2: Terminaison.

Prouver la terminaison d'un algorithme revient à prouver que pour **toute** entrée il termine.

Terminaison - 2

On se limite parfois aux entrées valides.

```
def f(a):  
    while a != 0:  
        a = a-1  
    return a
```

Termine sur toute entrée si on n'autorise pas a à être négatif. Il faudrait ici avoir donc comme pré-condition que a est un entier positifs.

Question : Que rajouter à ce code pour être sûr qu'on respecte la pré-condition ($a \geq 0$)?

Terminaison - 3

Pour prouver la terminaison d'une boucle (while) ou d'une fonction récursive, on utilise un variant.

Définition 3: Variant.

Un variant est une fonction des variables, à valeur dans $\mathbb{N}$ (ensemble des entiers positifs ou nuls) qui **décroît strictement** à chaque :

- itération de la boucle pour les algorithmes itératifs
- appel récursif pour les algorithmes récursifs

Autrement dit, on doit toujours se rapprocher de la condition d'arrêt. L'ensemble $\mathbb{N}$ étant minoré par 0 (on ne peut pas descendre en dessous), on ne peut pas décroître strictement indéfiniment.

Terminaison - 4

Reprenons notre exemple :

```
def pgcd(a, b):  
    while min(a,b) > 0:  
        if a < b:  
            b = b-a  
        else:  
            a = a-b  
    return max(a,b)
```

Question : Que peut-on choisir comme variant ?

Terminaison - 5

Pour notre fonction pgcd, on peut choisir comme variant $a + b$.

En effet à chaque tour de boucle :

- si $a < b$, alors b devient $b - a$,
donc $a + b$ devient $a + (b - a) = b$ (on décroît car $a > 0$).
- si $a \geq b$, alors a devient $a - b$,
donc $a + b$ devient $(a - b) + b = a$ (on décroît car $b > 0$).

Dans les deux cas, $a + b$ diminue strictement et reste supérieur ou égal à 0 (sinon on sort de la boucle).

Notre algorithme termine donc pour toute entrée valide.

Terminaison - 6

Limites

Parfois, la valeur étudiée peut alternativement croître et décroître.

```
def syracuse(n):  
    while n > 1:  
        if n % 2 == 0:  
            n = n // 2  
        else:  
            n = 3 * n + 1  
    return n
```

Dans ce cas là il n'est pas possible de trouver un variant et on ne sait pas si l'algorithme termine pour toute entrée. C'est d'ailleurs un problème ouvert en mathématiques.

Terminaison - 7

Généralisation : Ensembles bien fondés

Définition 4: Ordre bien fondé.

Un ordre bien fondé est un ordre sur un ensemble de données qui garantit qu'il n'existe aucune suite infinie strictement décroissante.

Propriété 1: Généralisation.

La notion de variant à valeurs dans $\mathbb{N}$ peut être généralisée à tout **ensemble** de données muni d'un **ordre bien fondé**.

Terminaison - 8

Exemple : Ordre lexicographique sur $\mathbb{N} \times \mathbb{N}$

Par exemple, on peut utiliser un tuple de variables (m, n) sur $\mathbb{N} \times \mathbb{N}$ muni de **l'ordre lexicographique** (l'ordre du dictionnaire).

On garantit la décroissance stricte à chaque récursion/itération si :

- Soit la valeur de m diminue strictement.
- Soit m reste constant, et alors la valeur de n doit diminuer strictement.

L'ordre lexicographique est bien fondé, ce qui garantit que le tuple ne pourra baisser indéfiniment, et donc que l'algorithme termine à tous les coups, même si un simple variant dans $\mathbb{N}$ ne suffisait pas.

Correction partielle itératifs

Un algorithme est dit **correct** si pour toute entrée, il produit une sortie correcte. Plus précisément, on parle de **correction partielle** lorsqu'on prouve que la sortie est correcte **si l'algorithme termine**.

Correction Totale = Terminaison + Correction Partielle.

Correction partielle itératifs - 2

Définition 5: Invariant de boucle.

Pour prouver la correction d'une boucle (while ou for), on utilise un **invariant de boucle**. C'est une propriété qui dépend des variables du programme, et qui vérifie trois conditions fondamentales :

1. **Initialisation** : L'invariant est vrai avant le premier passage dans la boucle.
2. **Conservation (Hérédité)** : S'il est vrai au début d'une itération et que la condition de la boucle est satisfaite, alors l'invariant reste vrai à la fin de cette itération.
3. **Sortie (Conclusion)** : À la sortie de la boucle, l'invariant est vrai, et la condition de la boucle est **fausse**. Leur conjugaison doit permettre de déduire la post-condition (la sortie attendue).

Correction partielle itératifs - 3

Exemple complet

Prouvons que la fonction suivante calcule bien la factorielle, avec comme invariant $I : r = i!$

```
def fact(n):  
    r = 1  
    i = 0  
    while i < n:  
        # invariant : r = i!  
        i = i + 1  
        r = r * i  
    return r
```


Correction partielle itératifs - 4

- **Initialisation :**

Avant la boucle, $r = 1$ et $i = 0$. On a bien $1 = 0!$, donc I est vrai.

- **Conservation :**

Supposons I vrai à l'entrée d'une itération et $i < n$.

Les nouvelles valeurs sont $i' = i + 1$ et $r' = r \times i'$.

$$\begin{aligned} r' &= r \times i' \\ &= i! \times (i + 1) \\ &= (i + 1)! \\ &= i'! \end{aligned}$$

L'invariant est conservé.

Correction partielle itératifs - 5

- **Conclusion :**

En sortie, l'invariant est vrai ($r = i!$) et la condition fausse ($i \geq n$).

Comme on incremente i de 1 jusqu'à n , on a exactement $i = n$.
On en déduit que $r = n!$, ce qui est bien le résultat de notre spécification.

Correction des algorithmes récursifs

Pour prouver la correction d'un algorithme récursif, on utilise généralement une preuve par **récurrence** (ou induction), qui s'appuie directement sur la structure des appels récursifs et sur la "taille" du problème.

Correction des algorithmes récursifs - 2

Le principe est le même que pour une récurrence mathématique classique : on définit la propriété à prouver $P(n)$ décrivant formellement ce que fait l'algorithme sur ses paramètres (ici n est un paramètre, il peut y en avoir plusieurs), puis on procède en deux étapes :

1. **Cas de base (Initialisation)** : On prouve que la spécification est respectée pour le(s) cas de base (lorsque la fonction ne fait pas d'appel récursif).
2. **Hérédité (Induction)** : On suppose que la fonction est correcte pour toute taille de problème "plus petite". Sous cette hypothèse d'induction, on démontre que le calcul de la fonction pour l'appel courant donne également un résultat valide.

Correction des algorithmes récurifs - 3

Exemple : Factorielle récursive

Prouvons que la fonction suivante calcule bien $n!$ pour tout $n \geq 0$.

```
def fact_rec(n):  
    # pre-condition : n >= 0  
    if n == 0:  
        return 1  
    else:  
        return n * fact_rec(n - 1)
```

Question : Donner la propriété $P(n)$ et prouver la correction de cet algorithme.

Correction des algorithmes récursifs - 4

On définit la propriété $P(n) : \text{fact_rec}(n) = n!$.

- **Cas de base** ($n = 0$) :

$\text{fact_rec}(0)$ retourne 1. Or $0! = 1$. Donc $P(0)$ est vraie.

- **Hérédité** :

Soit $n > 0$. Supposons $P(n - 1)$ vraie, c'est-à-dire que

$\text{fact_rec}(n-1) = (n-1)!$.

L'algorithme retourne $n * \text{fact_rec}(n-1)$.

Avec l'hypothèse d'induction, on obtient $n \times (n - 1)! = n!$.

La fonction retourne bien $n!$, donc $P(n)$ est vraie.

- **Conclusion** : Par principe de récurrence, la fonction réalise
retourne bien $n!$ pour tout $n \geq 0$. (et termine trivialement via le
variant n)